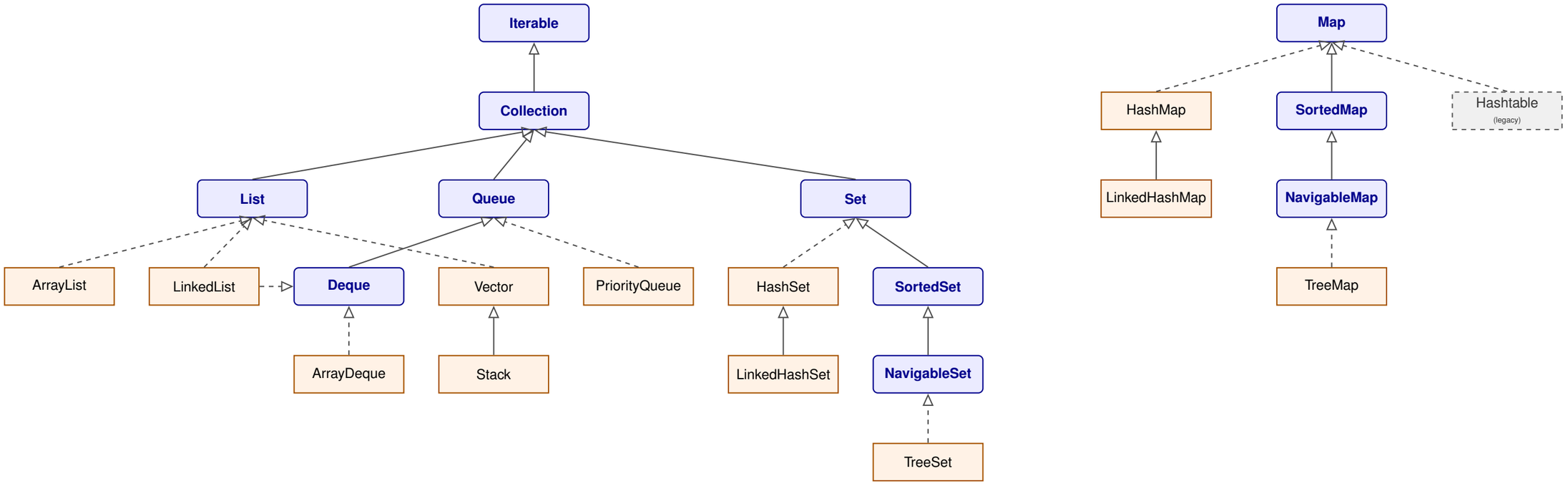
Java Collections Framework — Hierarchy (Corrected)

Solid arrow = *extends* Dashed arrow = *implements* **interface** class legacy class



Fixed vs. the original diagram: (1) “Dequeue” → *Deque*. (2) *Vector* implements *List* directly (not a child of *LinkedList*); *Stack* extends *Vector*, not *LinkedList*. (3) *LinkedList* implements both *List and Deque* — second link added. (4) Inserted missing *NavigableSet* (SortedSet → NavigableSet → TreeSet) and *NavigableMap* (SortedMap → NavigableMap → TreeMap). (5) *Hashtable* is a legacy class implementing *Map* directly (via *Dictionary*) — it is *not* a superclass of *HashMap*/*LinkedHashMap* as the dashed lines previously implied.

Java Collections Framework — Cheat Sheet

Core interfaces

- **Iterable** → **Collection** → List / Set / Queue
- **List**: ordered, indexed, duplicates OK
- **Set**: no duplicates, no index
- **Queue**: FIFO; **Deque** extends Queue (both ends)
- **Map**: key→value, *not* a Collection subtype
- **SortedSet**/**SortedMap**: sorted by key; **NavigableSet**/**NavigableMap** add floor/ceiling/higher/lower

Internal structures

Class	Internal structure
ArrayList	Resizable array
LinkedList	Doubly linked list (List+Deque)
Vector	Resizable array, synchronized
Stack	Extends Vector (LIFO)
ArrayDeque	Resizable circular array
PriorityQueue	Binary heap (min-heap default)
HashSet	Backed by HashMap
LinkedHashSet	HashMap + linked list
TreeSet	Red–Black tree
HashMap	Hash table (bucket, tree on collision)
LinkedHashMap	Hash table + linked list
TreeMap	Red–Black tree
Hashtable	Hash table, synchronized, legacy

Time complexity

Class	Search	Add	Remove
ArrayList	O(n)	O(1)*	O(n)
LinkedList	O(n)	O(1)†	O(1)†
ArrayDeque	O(n)	O(1)	O(1)
PriorityQueue	O(n)	O(log n)	O(log n)
HashSet/Map	O(1)	O(1)	O(1)
LinkedHashSet/Map	O(1)	O(1)	O(1)
TreeSet/Map	O(log n)	O(log n)	O(log n)

*amortized, at end. †at ends, or given a node reference; locating by index/value is O(n).

Ordering guarantee

- ArrayList/LinkedList/Vector: insertion order (indexed)
- LinkedHashSet/Map: insertion order (or access order if configured)
- TreeSet/Map: sorted (natural or Comparator)
- HashSet/Map, Hashtable: no guaranteed order

Null handling

- ArrayList, LinkedList, HashSet, HashMap, LinkedHashMap: allow nulls (HashMap: 1 null key, many null values)
- TreeSet/TreeMap: no null (throws NPE on natural ordering)
- Vector, Hashtable: no nulls (legacy, synchronized)
- ArrayDeque: no nulls

Thread safety

- Legacy & synchronized: Vector, Stack, Hashtable
- Modern collections (ArrayList, HashMap, etc.) are not synchronized
- Wrap with `Collections.synchronizedList/Map(...)`, or use `java.util.concurrent` (`ConcurrentHashMap`, `CopyOnWriteArrayList`)

Iterators

- Most collections: fail-fast (throw `ConcurrentModificationException` on structural change during iteration)
- `ConcurrentHashMap`, `CopyOnWriteArrayList`: fail-safe (iterate over a snapshot)
- Use `Iterator.remove()` to modify safely while iterating

Utility classes (not data structures)

- **Arrays**: sort, binarySearch, copyOf, fill, asList
- **Collections**: sort, reverse, shuffle, max, min, unmodifiableX, synchronizedX

When to use what

- Random access by index → ArrayList
- Frequent insert/delete at ends → ArrayDeque (preferred over Stack/LinkedList)
- Fast lookup, no order → HashMap/HashSet
- Need insertion order → LinkedHashMap/Set (also basis for LRU cache)
- Need sorted order → TreeMap/TreeSet
- Top-k / scheduling → PriorityQueue